INFORME DE LA PRUEBA PRÁCTICA UNIDAD 2

GRUPO H

Integrantes:

- Gualle Aucanshala Abisag Lisenia
- Guanotoa Escobar Karla Leonela
- Monar Parco Jhair Alexander

Nivel y Paralelo: Primero B

Asignatura: Algoritmos y Lógica de Programación

Docente: Ing. José Caiza **Fecha:** 14 de abril de 2026

Link del repositorio: https://github.com/Jhairmonar/Examen-Practico-Primer-Parcial

1. Objetivo General:

Desarrollar un sistema de gestión y control vehicular para la zona de parqueo de la UTA, mediante el uso de estructuras de datos avanzadas, algoritmos de ordenamiento y recursividad en el lenguaje C++ y su representación en pseudocódigo, con el fin de optimizar el registro, búsqueda y reporte de infracciones dentro del perímetro universitario.

1.1. Objetivos Específicos:

- Implementar la Programación Orientada a Objetos (POO) mediante la creación de la clase (Vehiculo_Campus), asegurando el encapsulamiento estricto de los datos y el uso de constructores sobrecargados para una inicialización eficiente de las unidades.
- Diseñar una función de búsqueda secuencial por placa para el rastreo de unidades y un algoritmo de ordenamiento por el método de "Burbuja" para jerarquizar a los conductores con mayor número de infracciones.
- Utilizar funciones recursivas para el cálculo del impacto total de infracciones en el sector y estructuras de control dinámicas para el módulo de entrenamiento (juego del ahorcado), fortaleciendo la interactividad del sistema.

2. Problema a resolver: Parqueadero universitario y control de infracciones

Contexto del caso: Registrar ingresos, salidas, tiempos de permanencia, cupos disponibles, tarifas y sanciones.

Clases mínimas sugeridas: Vehiculo_Campus y Registro_Parqueo

Requerimientos funcionales obligatorios

- Registrar vehículos con placa, propietario, tipo y carrera o departamento.
- Registrar entradas y salidas con hora, bloque y espacio asignado.
- Calcular permanencia, tarifa o gratuidad según tipo de usuario y detectar exceso.
- Buscar por placa y propietario, ordenar manualmente por mayor ocupación y más infracciones.
- Generar reportes diarios de ocupación y sanciones.

2.1. Análisis del Problema:

Este sistema automatiza la gestión integral de un parqueadero universitario y el control de las infracciones cometidas por los usuarios. El sistema debe permitir el registro completo de los vehículos (identificando propietario y pertenencia institucional como carrera o departamento), documentar cada evento de ingreso y salida (especificando hora, bloque y espacio exacto de estacionamiento), calcular la permanencia y aplicar la tarifa o gratuidad correspondiente según el perfil del usuario (estudiante, docente, administrativo), detectando excesos de tiempo para aplicar sanciones.

Además, debe posibilitar búsquedas avanzadas (por placa o propietario) y ordenamientos manuales por nivel de ocupación de bloques y cantidad de infracciones registradas. Finalmente, se debe generar un informe diario consolidado de ocupación y sanciones aplicadas, e integrar obligatoriamente un minijuego de "Ahorcado" que utilice placas simbólicas, bloques y tipos de infracciones frecuentes como datos de entrada.

Variables de entrada, proceso y salida.

- **Entrada:**
  - **Vehículo:** Placa (cadena), Propietario (cadena), Tipo de vehículo (cadena - ej: auto, moto, bicicleta), Carrera o Departamento (cadena).
  - **Ingreso:** Hora de ingreso (entero o tiempo), Bloque (cadena), Espacio (entero/cadena).

- **Salida:** Hora de salida (entero o tiempo).
- **Usuario:** Tipo de usuario (cadena - ej: estudiante, docente, administrativo, externo).
- **Parámetros:** Tabla de tarifas, límite de tiempo de permanencia, lista de sanciones y valores.
- **Para el Ahorcado:** Base de datos de palabras/términos (placas, bloques, infracciones).

- **Proceso:**
  - Registrar un nuevo vehículo con todos sus datos asociados.
  - Registrar la entrada de un vehículo, asignándole bloque y espacio.
  - Registrar la salida de un vehículo y calcular el tiempo transcurrido (permanencia).
  - Determinar la tarifa o gratuidad aplicable basándose en la permanencia y el tipo de usuario.
  - Verificar si el tiempo de permanencia excede el límite permitido para detectar excesos y aplicar la sanción correspondiente.
  - Realizar búsquedas de vehículos por placa o propietario.
  - Permitir ordenar manualmente los bloques por nivel de ocupación y cantidad de infracciones.
  - Generar reportes diarios detallando la ocupación total y las sanciones del día.
  - Construir la lógica del juego "Ahorcado" utilizando la base de datos de términos definidos.

- **Salida:**
  - Confirmación de registro de vehículo/entrada/salida exitosa.
  - Detalle del cálculo de permanencia y tarifa a pagar.
  - Notificación de sanción aplicada (si corresponde).
  - Listado de resultados de búsquedas (vehículos/propietarios).

- Vistas de bloques ordenadas por ocupación e infracciones.
- Reportes diarios consolidados.
- Interfaz y resultados del juego "Ahorcado".

2.2. Algoritmo:

1. Inicio
2. Cargar parámetros: Cargar tabla de tarifas, límites de tiempo, lista de sanciones y base de datos para el Ahorcado.
3. Inicializar: Estructuras de datos para registros de vehículos, ingresos/salidas y reportes diarios.
4. Menú principal: Presentar un menú con las opciones disponibles:
    - Registrar vehículo.
    - Registrar entrada de vehículo.
    - Registrar salida de vehículo y calcular tarifa/sanción.
    - Buscar vehículo/propietario.
    - Generar reportes diarios.
    - Jugar Ahorcado.
    - Salir.
5. Procesar opción: Ejecutar la acción correspondiente según la selección del usuario.
    - Si es "Registrar vehículo": Solicitar placa, propietario, tipo y carrera/departamento. Validar que no exista. Almacenar el registro.
    - Si es "Registrar entrada": Solicitar placa. Validar que no esté ya dentro. Solicitar hora, bloque y espacio. Validar disponibilidad. Almacenar el ingreso.
    - Si es "Registrar salida": Solicitar placa. Validar que esté dentro. Solicitar hora de salida.
        - Calcular permanencia.
        - Determinar tarifa/gratuidad según permanencia y tipo de usuario (estudiante, etc.).
        - Verificar si hay exceso de tiempo. Si lo hay, calcular y aplicar sanción.
        - Mostrar resumen de pago (tarifa + sanción). Actualizar registro.
    - Si es "Buscar vehículo/propietario": Solicitar criterio de búsqueda (placa o propietario). Mostrar los resultados encontrados. Ofrecer ordenamiento manual.

- Si es "Generar reportes diarios": Recopilar datos del día. Generar reporte consolidado de ocupación y lista de sanciones aplicadas.
- Si es "Jugar Ahorcado":
    - Seleccionar un término aleatorio (placa, bloque, infracción) de la base de datos.
    - Mostrar el tablero inicial (guiones bajos).
    - Iniciar ciclo de juego: solicitar letras, validar, actualizar tablero, controlar intentos.
    - Al finalizar el juego, mostrar el resultado (ganar/perder) y ofrecer jugar de nuevo o volver al menú.

6. Repetir: Volver al paso 4 hasta que la opción seleccionada sea "Salir".
7. Fin

2.3. Pseudocódigo:

```
Proceso SistemaControlZonaUTA
   // Configuración inicial (Simulación de constantes globales)
   Definir MAX_VEHICULOS Como Entero
   Definir TARIFA_HORA Como Real
   MAX_VEHICULOS <- 50
   TARIFA_HORA <- 1.50

   // Arreglos paralelos (Simulan la clase VehiculoCampus)
   Definir placas, propietarios, tipos, departamentos Como
   Caracter Definir horasPermanencia, infracciones Como Entero
   Dimension placas[50], propietarios[50], tipos[50], departamentos[50]
   Dimension horasPermanencia[50], infracciones[50]

   Definir totalRegistrados, opc Como Entero
     totalRegistrados <- 0
     opc <- 0

   // Variables temporales para el Menú
   Definir p, pr, t, d, placaMov, placaBuscada Como Caracter
```

```
Definir entrada, salida, horasCalculadas, indice, totales, i Como Entero
Definir aPagar Como Real

Repetir
    Escribir ""
    Escribir
    "=========================================="
    Escribir "   SISTEMA DE CONTROL DE ZONA UTA (GRUPO H)  "
    Escribir
    "=========================================="
    Escribir "1. Registrar nueva unidad (Vehiculo)"
    Escribir "2. Registrar movimiento (Ingreso/Salida)"
    Escribir "3. Rastreo por Placa (Busqueda)"
    Escribir "4. Reporte de Hostiles (Ordenar por Infracciones)"
    Escribir "5. Resumen del Perimetro (Infracciones Totales - Recursividad)"
    Escribir "6. Modulo de Ahorcado"
    Escribir "7. Desconectar"
    Escribir "Seleccione una opcion: " Sin Saltar
    Leer opcion

    Si opcion == 1 Entonces
        Si totalRegistrados < MAX_VEHICULOS Entonces
            // Nota en PSeInt los arreglos inician en 1 (por defecto en perfil estricto)
            totalRegistrados <- totalRegistrados + 1

            Escribir "Placa del vehiculo (Ej: AAA123): " Sin Saltar
            Leer p
            Escribir "Nombre del propietario (Un nombre): " Sin Saltar
            Leer pr
            Escribir "Tipo (Auto/Moto): " Sin Saltar
            Leer t
            Escribir "Departamento/Carrera: " Sin Saltar
            Leer d
```

```
        placas[totalRegistrados] <- p
        propietarios[totalRegistrados] <- pr
        tipos[totalRegistrados] <- t
        departamentos[totalRegistrados] <- d
        horasPermanencia[totalRegistrados] <- 0
        infracciones[totalRegistrados] <- 0

        Escribir "[+] Unidad registrada
      exitosamente." Sino
        Escribir "[!] Capacidad de zona al maximo."
      FinSi

    SiNo Si opcion == 2 Entonces
                        Escribir "Ingrese placa para el registro de horas: " Sin
                        Saltar Leer placaMov

                        // PSeInt tiene la función Mayusculas() que reemplaza el ciclo
ASCII manual de
C++
                        placaMov <- Mayusculas(placaMov)

                        indice <- buscarVehiculo(placas, totalRegistrados, placaMov)

                        Si indice <> -1 Entonces
                              Escribir "Hora de entrada (Formato 24h, ej: 8): " Sin

Saltar
                              Leer entrada
                              Escribir "Hora de salida (Formato 24h, ej: 12): " Sin

Saltar
                              Leer salida

                              horasCalculadas <- calcularHoras(entrada, salida)
```

```
                    horasPermanencia[indice]
<-horasPermanencia[indice] + horasCalculadas

                    aPagar <- horasCalculadas * TARIFA_HORA
                    Escribir "[+] Movimiento registrado. Permanencia:
                    ",
horasCalculadas, "h. Tarifa: $", aPagar

                    Si horasCalculadas > 8 Entonces
                        infracciones[indice] <- infracciones[indice] + 1
                        Escribir "[!] ALERTA: Exceso de permanencia
detectado. Infraccion
asignada."
                    FinSi
                Sino
                    Escribir "[-] Objetivo no encontrado en el radar."
                FinSi

            SiNo Si opcion == 3 Entonces
                    Escribir "Placa a interceptar: " Sin Saltar
                    Leer placaBuscada
                    placaBuscada <- Mayusculas(placaBuscada)

                    indice <- buscarVehiculo(placas, totalRegistrados,
placaBuscada)

                    Si indice <> -1 Entonces
                        Escribir "\n--- DATOS DE LA UNIDAD ---"
                        Escribir "Placa: ", placas[indice], " |
Propietario: ", propietarios[indice], " | Tipo: ", tipos[indice], " | Depto: ",
departamentos[indice], " | Horas: ", horasPermanencia[indice], " | Infracciones: ",
infracciones[indice]
                    Sino
```

```
                    Escribir "[-] Objetivo evasivo. No encontrado."
                FinSi

            SiNo Si opcion == 4 Entonces
                    Si totalRegistrados > 0 Entonces
                        ordenarPorInfracciones(placas,
propietarios, tipos, departamentos, horasPermanencia, infracciones, totalRegistrados)
                        Para i <- 1 Hasta totalRegistrados Hacer
                            Escribir "Placa: ", placas[i], " |
Propietario: ", propietarios[i], " | Tipo: ", tipos[i], " | Depto: ", departamentos[i], " |
Horas: ", horasPermanencia[i], " | Infracciones: ", infracciones[i]
                        FinPara
                    Sino
                        Escribir "[-] No hay registros en el
perimetro."
                    FinSi

                SiNo Si opcion == 5 Entonces
                        totales
<-totalInfraccionesSect(infracciones, totalRegistrados)
                        Escribir "\n[REPORTE TACTICO]
Infracciones totales en el campus: ", totales

                    SiNo Si opcion == 6 Entonces
                            ejecutarAhorcado()
                        FinSi
                    FinSi
                FinSi
            FinSi
        FinSi
    FinSi
```

```
    Hasta Que opcion == 7

    Escribir "\n[!] Sistema desconectado. Fin de la transmision."
FinProceso

// ==========================================
// FUNCIONES Y SUBPROCESOS
// ==========================================

Funcion horas <- calcularHoras(entrada, salida)
    Definir horas Como Entero
    horas <- salida - entrada
    Si horas == 0 Entonces
        horas <- 1
    FinSi
 FinFuncio
        n

// Función RECURSIVA para sumar infracciones
Funcion total <- totalInfraccionesSect(infracciones, n)
    Definir total Como Entero
    Si n <= 0 Entonces
        total <- 0
    Sino
        total <- infracciones[n] + totalInfraccionesSect(infracciones, n - 1)
    FinSi
FinFuncion

// Búsqueda Secuencial
Funcion indice <- buscarVehiculo(placas, n, placaBuscada)
    Definir indice, i Como Entero
```

```
    indice <- -1
    Para i <- 1 Hasta n Hacer
        Si placas[i] == placaBuscada Entonces
            indice <- i
            i <- n // Romper el ciclo simulado
        FinSi
    FinPara
FinFuncion

// Ordenamiento Burbuja Adaptado a Arreglos Paralelos
SubProceso ordenarPorInfracciones(placas Por Referencia, propietarios Por Referencia, tipos Por Referencia, departamentos Por Referencia, horasPermanencia Por Referencia, infracciones Por Referencia, n)
    Definir i, j Como Entero
    Definir tempStr Como Caracter
    Definir tempInt Como Entero

    Para i <- 1 Hasta n - 1 Hacer
        Para j <- 1 Hasta n - i Hacer
            Si infracciones[j] < infracciones[j + 1] Entonces
                // Intercambio de Infracciones
                tempInt <- infracciones[j]
                infracciones[j] <- infracciones[j + 1]
                infracciones[j + 1] <- tempInt

                // Intercambio de Horas
                tempInt <-
                horasPermanencia[j]
                horasPermanencia[j] <- horasPermanencia[j + 1]
                horasPermanencia[j + 1] <- tempInt

                // Intercambio de Placas
```

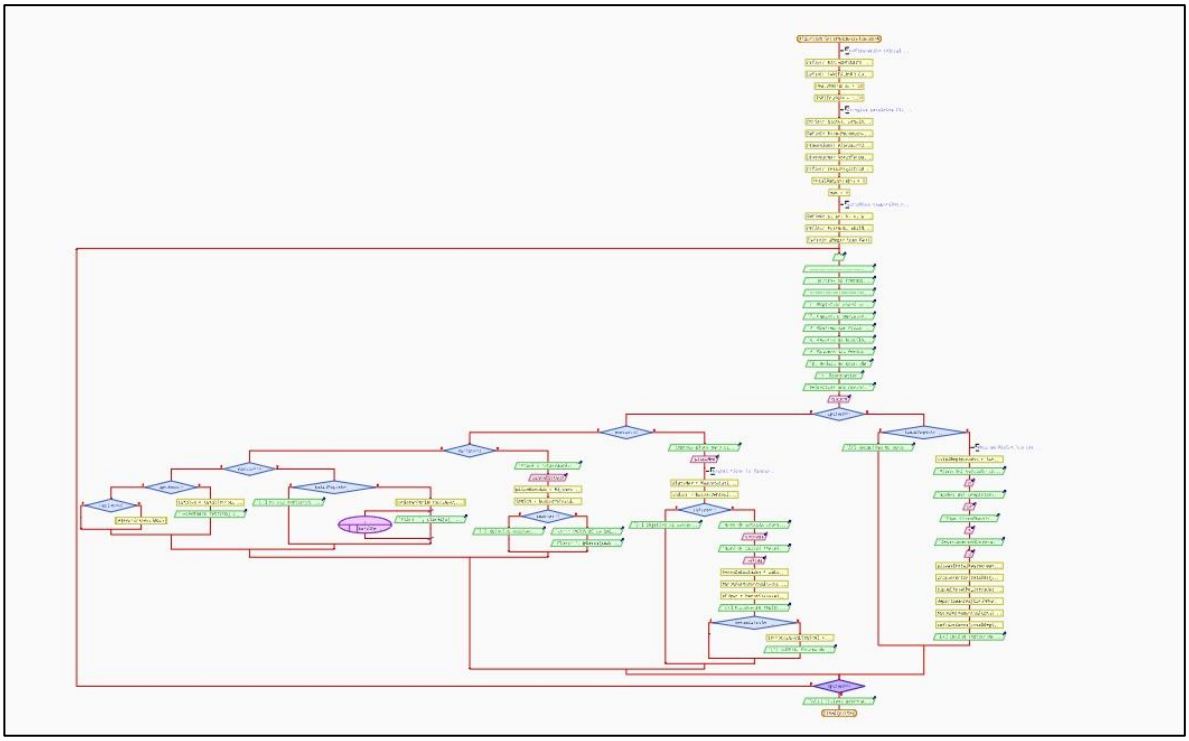
```
            tempStr <- placas[j]
            placas[j] <- placas[j + 1]
            placas[j + 1] <- tempStr

            // Intercambio de Propietarios
            tempStr <- propietarios[j]
            propietarios[j] <- propietarios[j +
            1] propietarios[j + 1] <- tempStr

            // Intercambio de Tipos
            tempStr <- tipos[j]
            tipos[j] <- tipos[j + 1]
            tipos[j + 1] <- tempStr

            // Intercambio de Departamentos
            tempStr <- departamentos[j]
            departamentos[j] <- departamentos[j +
            1] departamentos[j + 1] <- tempStr
        FinSi
      FinPar
      a
    FinPara
    Escribir "\n[!] Reordenamiento completado. Listando hostiles primero...\n"
FinSubProceso

// Módulo de Juego Ahorcado
SubProceso ejecutarAhorcado
    Definir palabraOculta, progreso, letra, nuevoProgreso Como
    Caracter Definir intentos, i Como Entero
    Definir victoria, acierto Como Logico

    palabraOculta <- "INFRACCION"
```

```
progreso <- "__________"
intentos <- 6
victoria <- Falso

Escribir
"\n=========================================
=" Escribir "  MODULO DE ENTRENAMIENTO: AHORCADO"
Escribir
"=========================================
"
Escribir "Mision: Descifrar el codigo del objetivo."

Mientras intentos > 0 Y NO victoria Hacer
   Escribir "\nEstado actual: ", progreso
   Escribir "Vidas restantes: ", intentos
   Escribir "Ingrese una letra: " Sin Saltar
   Leer letra

   letra <- Mayusculas(letra)
   acierto <- Falso
   nuevoProgreso <- ""

   // Reconstrucción del string (PSeInt no permite progreso[i] = letra
   directamente) Para i <- 1 Hasta Longitud(palabraOculta) Hacer
      Si Subcadena(palabraOculta, i, i) == letra Y Subcadena(progreso, i, i) == "_"
Entonces
         nuevoProgreso <- nuevoProgreso +
         letra acierto <- Verdadero
      Sino
         nuevoProgreso <- nuevoProgreso + Subcadena(progreso, i, i)
      FinSi
   FinPara

   progreso <- nuevoProgreso
```

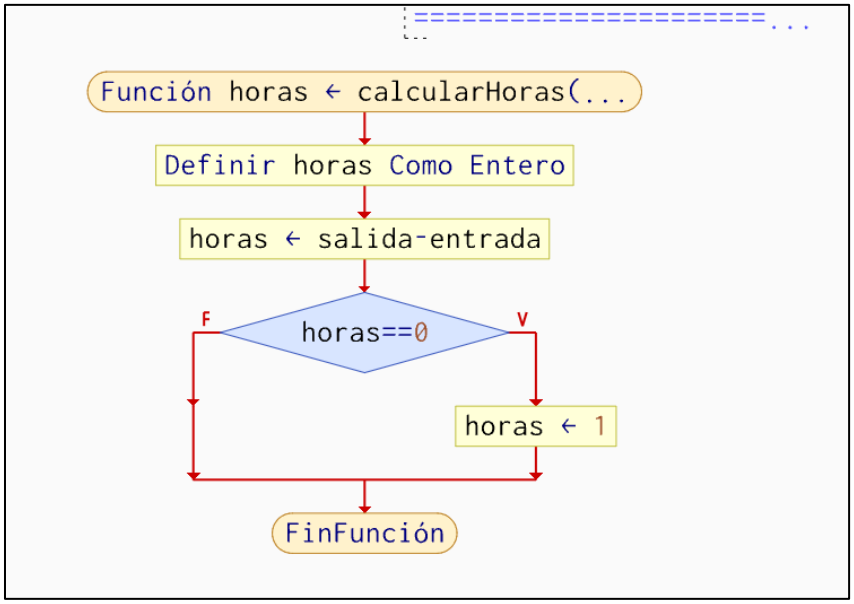
```
        Si NO acierto Entonces
            intentos <- intentos - 1
            Escribir "[-] Letra incorrecta. Recibiendo dano."
        Sino
            Escribir "[+] Acierto confirmado."
        FinSi

        Si progreso == palabraOculta Entonces
            victoria <- Verdadero
        FinSi
    FinMientras

    Si victoria Entonces
        Escribir "\n[VICTORIA] Codigo descifrado: ", palabraOculta, ". Mision
cumplida."
    Sino
        Escribir "\n[DERROTA] Intentos agotados. La palabra era: ", palabraOculta
    FinSi
FinSubProceso
```

2.4. Diagrama de Flujo:

- *Diagrama SistemaControlUTA*



- *Diagrama Función horas:*



2.5. Codigo en C++:

```
#include <iostream>
#include <string>
using namespace std;

// Constantes globales de operación
const int MAX_VEHICULOS = 50;
const double TARIFA_HORA = 1.50;

// ========================================
// CLASE 1: VehiculoCampus
// ========================================
class VehiculoCampus {
private:
string placa;
string propietario;
string tipo;
string departamento;
int horasPermanencia;
int infracciones;

public:
// Constructor: Inicializa la unidad en la base de datos
VehiculoCampus(string p = "", string prop = "", string t = "", string dep = "") {
placa = p;
propietario = prop;
tipo = t;
departamento = dep;
horasPermanencia = 0;
infracciones = 0;
}

// Getters: Extracción de datos clasificados
string getPlaca() { return placa; }
string getPropietario() { return propietario; }
```

```
int getHorasPermanencia() { return horasPermanencia; }
int getInfracciones() { return infracciones; }

// Setters y métodos de actualización operativa
void registrarMovimiento(int horas) {
horasPermanencia += horas;
}

void reportarInfraccion() {
infracciones++;
}

// Visualización del estado actual de la unidad
void mostrarDatos() {
cout << "Placa: " << placa << " | Propietario: " << propietario
<< " | Tipo: " << tipo << " | Depto: " << departamento
<< " | Horas: " << horasPermanencia << " | Infracciones: " << infracciones << endl;
}
};


// Depuración del fragmento asignado (Grupo H)
// Corrección táctica: Uso de '==' para comparación en lugar de '=' (asignación)
int calcularHoras(int entrada, int salida) {
int horas = salida - entrada;
if (horas == 0) {
return 1; // Tarifa mínima por despliegue corto
}
return horas;
}

// Función RECURSIVA: Calcula el impacto total (suma de infracciones) en el perímetro
int totalInfraccionesSect(VehiculoCampus vehiculos[], int n) {
if (n <= 0) return 0; // Caso base: Fin del barrido
```

```
return vehiculos[n - 1].getInfracciones() + totalInfraccionesSect(vehiculos, n - 1);
}

// Ordenamiento manual: Táctica de "Burbuja" para listar mayores infractores (Sin usar sort)
void ordenarPorInfracciones(VehiculoCampus vehiculos[], int n) {
for (int i = 0; i < n - 1; i++) {
for (int j = 0; j < n - i - 1; j++) {
if (vehiculos[j].getInfracciones() < vehiculos[j + 1].getInfracciones()) {
// Intercambio de posiciones
VehiculoCampus temp = vehiculos[j];
vehiculos[j] = vehiculos[j + 1];
vehiculos[j + 1] = temp;
}
}
}
cout << "\n[!] Reordenamiento completado. Listando hostiles primero...\n";
}

// Búsqueda manual: Rastreo secuencial por placa
int buscarVehiculo(VehiculoCampus vehiculos[], int n, string placaBuscada) {
for (int i = 0; i < n; i++) {
if (vehiculos[i].getPlaca() == placaBuscada) {
return i; // Objetivo localizado
}
}
return -1; // Objetivo no encontrado
}

// ========================================
// MÓDULO OBLIGATORIO: ENTRENAMIENTO (AHORCADO)
// ========================================
void ejecutarAhorcado() {
cout << "\n=====================================\n";
cout << "   MODULO DE ENTRENAMIENTO: AHORCADO\n";
```

```
cout << "=====================================\n";

// Vocabulario restringido al dominio del Grupo H (Parqueadero)
string palabraOculta = "INFRACCION";
string progreso = "__________";
int intentos = 6;
bool victoria = false;
char letra;

cout << "Mision: Descifrar el codigo del objetivo.\n";

while (intentos > 0 && !victoria) {
cout << "\nEstado actual: " << progreso << "\n";
cout << "Vidas restantes: " << intentos << "\n";
cout << "Ingrese una letra: ";
cin >> letra;

// CONVERSIÓN MANUAL A MAYÚSCULA (Lógica ASCII)
if (letra >= 'a' && letra <= 'z') {
letra = letra - 32;
}

bool acierto = false;
for (int i = 0; i < palabraOculta.length(); i++) {
if (palabraOculta[i] == letra && progreso[i] == '_') {
progreso[i] = letra;
acierto = true;
}
}

if (!acierto) {
intentos--;
cout << "[-] Letra incorrecta. Recibiendo dano.\n";
} else {
```

```
cout << "[+] Acierto confirmado.\n";
}

if (progreso == palabraOculta) {
victoria = true;
}
}

if (victoria) {
cout << "\n[VICTORIA] Codigo descifrado: " << palabraOculta << ". Mision cumplida.\n";
} else {
cout << "\n[DERROTA] Intentos agotados. La palabra era: " << palabraOculta << "\n";
}
}

// ========================================
// CENTRO DE MANDO
// ========================================
int main() {
VehiculoCampus baseDatos[MAX_VEHICULOS];
int totalRegistrados = 0;
int opcion;

do {
cout << "\n========================================\n";
cout << "  SISTEMA DE CONTROL DE ZONA UTA (GRUPO H)  \n";
cout << "========================================\n";
cout << "1. Registrar nueva unidad (Vehiculo)\n";
cout << "2. Registrar movimiento (Ingreso/Salida)\n";
cout << "3. Rastreo por Placa (Busqueda)\n";
cout << "4. Reporte de Hostiles (Ordenar por Infracciones)\n";
cout << "5. Resumen del Perimetro (Infracciones Totales - Recursividad)\n";
cout << "6. Modulo de Ahorcado\n";
cout << "7. Desconectar\n";
```

```
cout << "Seleccione una opcion: ";
cin >> opcion;

if (opcion == 1) {
if (totalRegistrados < MAX_VEHICULOS) {
string p, pr, t, d;
cout << "Placa del vehiculo (Ej: AAA123): "; cin >> p;
cout << "Nombre del propietario (Un nombre): "; cin >> pr;
cout << "Tipo (Auto/Moto): "; cin >> t;
cout << "Departamento/Carrera: "; cin >> d;

baseDatos[totalRegistrados] = VehiculoCampus(p, pr, t, d);
totalRegistrados++;
cout << "[+] Unidad registrada exitosamente.\n";
} else {
cout << "[!] Capacidad de zona al maximo.\n";
}
}
else if (opcion == 2) {
string placaMov;
cout << "Ingrese placa para el registro de horas: "; cin >> placaMov;

// CONVERSIÓN MANUAL A MAYÚSCULAS PARA LA PLACA INGRESADA
for (int i = 0; i < placaMov.length(); i++) {
if (placaMov[i] >= 'a' && placaMov[i] <= 'z') {
placaMov[i] = placaMov[i] - 32;
}
}

int indice = buscarVehiculo(baseDatos, totalRegistrados, placaMov);

if (indice != -1) {
int entrada, salida;
cout << "Hora de entrada (Formato 24h, ej: 8): "; cin >> entrada;
```

```
cout << "Hora de salida (Formato 24h, ej: 12): "; cin >> salida;

int horasCalculadas = calcularHoras(entrada, salida);
baseDatos[indice].registrarMovimiento(horasCalculadas);

double aPagar = horasCalculadas * TARIFA_HORA;
cout << "[+] Movimiento registrado. Permanencia: " << horasCalculadas << "h. Tarifa: $" <<
aPagar << "\n";

if (horasCalculadas > 8) {
baseDatos[indice].reportarInfraccion();
cout << "[!] ALERTA: Exceso de permanencia detectado. Infraccion asignada al objetivo.\n";
}
} else {
cout << "[-] Objetivo no encontrado en el radar.\n";
}
}
else if (opcion == 3) {
string placaBuscada;
cout << "Placa a interceptar: "; cin >> placaBuscada;

// CONVERSIÓN MANUAL A MAYÚSCULAS PARA LA BÚSQUEDA
for (int i = 0; i < placaBuscada.length(); i++) {
if (placaBuscada[i] >= 'a' && placaBuscada[i] <= 'z') {
placaBuscada[i] = placaBuscada[i] - 32;
}
}

int indice = buscarVehiculo(baseDatos, totalRegistrados, placaBuscada);
if (indice != -1) {
cout << "\n--- DATOS DE LA UNIDAD ---\n";
baseDatos[indice].mostrarDatos();
} else {
cout << "[-] Objetivo evasivo. No encontrado.\n";
```

```
}
}
else if (opcion == 4) {
if (totalRegistrados > 0) {
ordenarPorInfracciones(baseDatos, totalRegistrados);
for (int i = 0; i < totalRegistrados; i++) {
baseDatos[i].mostrarDatos();
}
} else {
cout << "[-] No hay registros en el perimetro.\n";
}
}
else if (opcion == 5) {
int totales = totalInfraccionesSect(baseDatos, totalRegistrados);
cout << "\n[REPORTE TACTICO] Infracciones totales en el campus: " << totales << "\n";
}
else if (opcion == 6) {
ejecutarAhorcado();
}

} while (opcion != 7);

cout << "\n[!] Sistema desconectado. Fin de la transmision.\n";
return 0;
}
```

2.6. Pruebas de Solución:

Ilustración 1: Vista del menú del sistema

```
==========================================
   SISTEMA DE CONTROL DE ZONA UTA (GRUPO H)
==========================================
1. Registrar nueva unidad (Vehiculo)
2. Registrar movimiento (Ingreso/Salida)
3. Rastreo por Placa (Busqueda)
4. Reporte de Hostiles (Ordenar por Infracciones)
5. Resumen del Perimetro (Infracciones Totales - Recursividad)
6. Modulo de Ahorcado
7. Desconectar
Seleccione una opcion: |
```

Ilustración 2: Juego del Ahorcado implementado al programa

```
==========================================
   SISTEMA DE CONTROL DE ZONA UTA (GRUPO H)
==========================================
1. Registrar nueva unidad (Vehiculo)
2. Registrar movimiento (Ingreso/Salida)
3. Rastreo por Placa (Busqueda)
4. Reporte de Hostiles (Ordenar por Infracciones)
5. Resumen del Perimetro (Infracciones Totales - Recursividad)
6. Modulo de Ahorcado
7. Desconectar
Seleccione una opcion: 6

=====================================
   MODULO DE ENTRENAMIENTO: AHORCADO
=====================================
Mision: Descifrar el codigo del objetivo.

Estado actual: __________
Vidas restantes: 6
Ingrese una letra: i
[+] Acierto confirmado.

Estado actual: I_____I__
Vidas restantes: 6
Ingrese una letra: |
```

Ilustración 3: Registro de un nuevo vehiculo

```
==========================================
   SISTEMA DE CONTROL DE ZONA UTA (GRUPO H)
==========================================
1. Registrar nueva unidad (Vehiculo)
2. Registrar movimiento (Ingreso/Salida)
3. Rastreo por Placa (Busqueda)
4. Reporte de Hostiles (Ordenar por Infracciones)
5. Resumen del Perimetro (Infracciones Totales - Recursividad)
6. Modulo de Ahorcado
7. Desconectar
Seleccione una opcion: 1
Placa del vehiculo (Ej: AAA123): ASE123
Nombre del propietario (Un nombre): Juan Lopez
Tipo (Auto/Moto): Departamento/Carrera: Chevrolet
[+] Unidad registrada exitosamente.
```

Ilustración 4: Búsqueda de la unidad registrada

```
=========================================
   SISTEMA DE CONTROL DE ZONA UTA (GRUPO H)
=========================================
1. Registrar nueva unidad (Vehiculo)
2. Registrar movimiento (Ingreso/Salida)
3. Rastreo por Placa (Busqueda)
4. Reporte de Hostiles (Ordenar por Infracciones)
5. Resumen del Perimetro (Infracciones Totales - Recursividad)
6. Modulo de Ahorcado
7. Desconectar
Seleccione una opcion: 3
Placa a interceptar: ASE123

--- DATOS DE LA UNIDAD ---
Placa: ASE123 | Propietario: Juan | Tipo: Lopez | Depto: Chevrolet | Horas: 0 | Infracciones: 0
```

3. Conclusiones:

En conclusión, el sistema que fue desarrollado ha demostrado ser una herramienta eficaz para poder controlar administrativamente un parqueadero universitario, ya que adicionalmente a implementar encapsulamiento y validaciones de entrada, se permite que la información (horas de permanencia, infracciones, etc.) no se cuarte y se centraliza al máximo, evitando cualquier error del usuario.

Esa misma propiedad implementativa (del código a pseudocódigo) donde se observa el cambio que se genera de Clases a Arreglos Paralelos, sugiere que la lógica algorítmica se conserva en su forma y los mecanismos de control (perímetro) también lo hacen, indicando que se tienen buenos sistemas de ordenamiento y escalabilidad del perímetro.

4. Recomendaciones:

- Validación de tipos de datos: Para futuras versiones, se recomienda incluir un manejo de excepción eso validaciones adicionales para evitar que el programa falle si el usuario ingresa letras en campos que esperan números (como en la tarifa o las horas).

- Persistencia de datos: Actualmente, los datos se pierden al cerrar el programa. Se sugiere integrar el manejo de archivos de texto o binarios para que la base de datos de vehículos se mantenga guardada permanentemente.

- Interfaz Gráfica: Aunque el sistema funciona perfectamente por consola, se recomienda explorar librerías gráficas para mejorar la experiencia del usuario, facilitando la visualización de los "hostiles" o infractores mediante una tabla más legible.